



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Práctica 3:

Circuitos de 1 y 2 orden.

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Microcomputadoras

Grupo: 03 - Semestre: 2026-2

Calf: _____



1. Objetivo:

2. Desarrollo:

Los parámetros del sistema son:

- **Frecuencia de muestreo:** $f_s = 8000$ Hz
- **Período de muestreo:** $T = \frac{1}{f_s} = \frac{1}{8000} = 1.25 \times 10^{-4} \text{ s} = 125 \mu\text{s}$
- **Frecuencia de Nyquist:** $f_N = \frac{f_s}{2} = 4000$ Hz
- **Entrada analógica:** GP26 (ADC0)
- **Generación de señal de prueba:** GP3 (PWM, onda cuadrada)

Para el uso de los hilos en la pico: el **Núcleo 1** ejecuta la ecuación en diferencias del filtro activo con temporización precisa a $f_s = 8000$ Hz, mientras que el **Núcleo 0** procesa todas las entradas que se van recibiendo.

2.1. Filtro Pasa Bajas de Primer Orden (IIR Butterworth)

Se selecciona una frecuencia de corte $f_c \approx 79.6$ Hz para el filtro pasa bajas. Esto por:

1. **Criterio de Nyquist:** La frecuencia de corte debe satisfacer $f_c < f_N = 4000$ Hz, con $f_c \approx 79.6$ Hz.
2. **Relación f_c/f_s :** La relación $f_c/f_s = 79.6/8000 = 0.00995$ es muy pequeña; la distorsión por frequency warping es despreciable y se corrige con la pre-distorsión.
3. **Consistencia con el código:** Los coeficientes digitales implementados ($A_0 = A_1 = 0.0303$, $B_1 = 0.9394$) implican $\Omega_a = 0.03125$, lo cual fija f_c en aproximadamente 79.6 Hz.

Circuito eléctrico

El filtro pasa bajas analógico de primer orden se implementa con un circuito RC serie, donde la salida se toma sobre el capacitor:

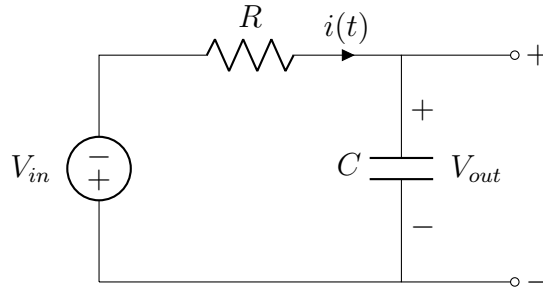


Figura 1: Circuito RC pasa bajas de primer orden.

calculo de componentes:

La frecuencia de corte del circuito RC está dada por:

$$f_c = \frac{1}{2\pi RC} \implies \omega_c = \frac{1}{RC} = 2\pi f_c \quad (1)$$

Seleccionando $C = 100 \text{ nF}$ (serie E24), se calcula la resistencia necesaria para obtener $f_c \approx 79.6 \text{ Hz}$:

$$R = \frac{1}{2\pi f_c C} = \frac{1}{2\pi(79.6)(100 \times 10^{-9})} = \frac{1}{5.000 \times 10^{-5}} = 20000 \Omega \approx 20.0 \text{ k}\Omega \quad (2)$$

Se utiliza una resistencia de $R = 20 \text{ k}\Omega$ (serie E24), lo cual resulta en una frecuencia de corte real de:

$$f_{c,\text{real}} = \frac{1}{2\pi(20000)(100 \times 10^{-9})} = 79.6 \text{ Hz} \quad (3)$$

La diferencia es nula a nivel de redondeo.

2.1.1. Obtención de la función de transferencia $H(s)$

Aplicando la ley de Kirchhoff al circuito RC en el dominio de Laplace:

$$V_{in}(s) = R \cdot I(s) + \frac{1}{sC} \cdot I(s) = I(s) \left(R + \frac{1}{sC} \right) \quad (4)$$

El voltaje de salida sobre el capacitor es:

$$V_{out}(s) = \frac{1}{sC} \cdot I(s) \quad (5)$$



Despejando la corriente de la primera ecuación y sustituyendo:

$$H(s) = \frac{V_{out}(s)}{V_{in}(s)} = \frac{\frac{1}{sC}}{R + \frac{1}{sC}} = \frac{1}{sRC + 1} \quad (6)$$

Multiplicando numerador y denominador por $\omega_c = 1/(RC)$:

$$H(s) = \frac{\omega_c}{s + \omega_c}, \quad \omega_c = 2\pi(79.6) = 500.0 \text{ rad/s} \quad (7)$$

Esta es la función de transferencia de un filtro Butterworth pasa bajas de primer orden, con ganancia unitaria en DC ($H(0) = 1$) y atenuación de -3 dB en $\omega = \omega_c$.

2.1.2. Discretización mediante la transformada bilineal

Paso 1: Pre-distorsión de frecuencia. La transformada bilineal introduce una distorsión no lineal entre las frecuencias analógica y digital. Para garantizar que la frecuencia de corte sea exactamente la misma y se mantenga, se calcula la frecuencia analógica pre-distorsionada:

$$\Omega_a = \tan\left(\frac{\pi f_c}{f_s}\right) = \tan\left(\frac{\pi \cdot 79.6}{8000}\right) = \tan(0.03124 \text{ rad}) \quad (8)$$

Evalutando:

$$\Omega_a = 0.03125 \quad (9)$$

$$H(s) = \frac{\Omega_a}{s + \Omega_a} \quad (10)$$

Paso 2: Sustitución de la variable bilineal.

$$s = \frac{z - 1}{z + 1} \quad (11)$$

Sustituyendo en $H(s)$:

$$H(z) = \frac{\Omega_a}{\frac{z - 1}{z + 1} + \Omega_a} \quad (12)$$

simplificando:



Multiplicando numerador y denominador por $(z + 1)$:

$$H(z) = \frac{\Omega_a(z + 1)}{(z - 1) + \Omega_a(z + 1)} \quad (13)$$

Expandiendo el denominador:

$$\begin{aligned} (z - 1) + \Omega_a(z + 1) &= z - 1 + \Omega_a z + \Omega_a \\ &= z(1 + \Omega_a) + (\Omega_a - 1) \end{aligned} \quad (14)$$

Dividiendo numerador y denominador entre $(1 + \Omega_a)$:

$$H(z) = \frac{\frac{\Omega_a}{1 + \Omega_a} (z + 1)}{z + \frac{\Omega_a - 1}{1 + \Omega_a}} = \frac{\frac{\Omega_a}{1 + \Omega_a} (z + 1)}{z - \frac{1 - \Omega_a}{1 + \Omega_a}} \quad (15)$$

Paso 3: Identificación con la forma estándar. Comparando con la forma canónica

$$H(z) = \frac{A_0 z + A_1}{z - B_1}:$$

$$A_0 = A_1 = \frac{\Omega_a}{1 + \Omega_a} = \frac{0.03125}{1 + 0.03125} = \frac{0.03125}{1.03125} = 0.0303 \quad (16)$$

$$B_1 = \frac{1 - \Omega_a}{1 + \Omega_a} = \frac{1 - 0.03125}{1 + 0.03125} = \frac{0.96875}{1.03125} = 0.9394 \quad (17)$$

Por lo tanto, la función de transferencia discreta es:

$$\boxed{H(z) = \frac{0.0303 (z + 1)}{z - 0.9394} = \frac{0.0303 z + 0.0303}{z - 0.9394}} \quad (18)$$

2.1.3. Ecuación en diferencias

A partir de $H(z) = Y(z)/U(z)$:

$$Y(z) (z - B_1) = U(z) (A_0 z + A_1) \quad (19)$$

Dividiendo ambos lados entre z :

$$Y(z) - B_1 z^{-1} Y(z) = A_0 U(z) + A_1 z^{-1} U(z) \quad (20)$$

Aplicando la transformada z inversa (donde z^{-1} representa un retardo de una muestra):

$$y(k) = A_0 \cdot u(k) + A_1 \cdot u(k-1) + B_1 \cdot y(k-1) \quad (21)$$

Sustituyendo los valores numéricos:

$$y(k) = 0.0303 \cdot u(k) + 0.0303 \cdot u(k-1) + 0.9394 \cdot y(k-1) \quad (22)$$

Coefficientes:

Tabla 1: Coeficientes del filtro pasa bajas de primer orden ($f_c \approx 79.6$ Hz, $f_s = 8000$ Hz).

Coeficiente	Expresión	Valor numérico
A_0	$\frac{\Omega_a}{1 + \Omega_a}$	0.0303
A_1	$\frac{\Omega_a}{1 + \Omega_a}$	0.0303
A_2	—	0.0
B_1	$\frac{1 - \Omega_a}{1 + \Omega_a}$	0.9394
B_2	—	0.0

Verificación en DC ($z = 1$):

$$H(1) = \frac{A_0 + A_1}{1 - B_1} = \frac{0.0303 + 0.0303}{1 - 0.9394} = \frac{0.0606}{0.0606} = 1.0 \quad (23)$$

De este modo, se verifica que la componente DC no sufre atenuación.

2.1.4. Diagrama de flujo de señal

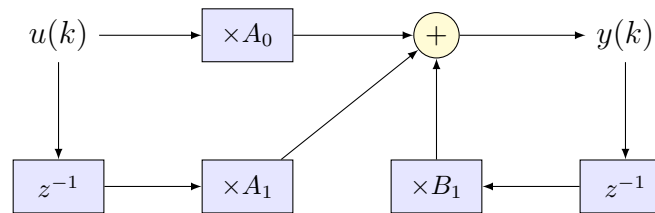


Figura 2: Diagrama de flujo de señal del filtro pasa bajas de primer orden (Forma Directa I).

2.2. Diseño del Filtro Pasa Altas (Circuito RC)

Se selecciona una frecuencia de corte $f_c = 500$ Hz para el filtro pasa altas. Para esto, se tomo en cuenta lo siguiente:

1. **Criterio de Nyquist:** Se cumple $f_c = 500$ Hz $< f_N = 4000$ Hz.
2. **Relación:** La relación $f_c/f_s = 500/8000 = 0.0625$ se ubica en el rango recomendado, donde la transformada bilineal presenta mínima distorsión.
3. **Consistencia con el código:** Los coeficientes implementados ($A_0 = 0.8341$, $A_1 = -0.8341$, $B_1 = 0.6682$) corresponden exactamente a $f_c = 500$ Hz.

Circuito:

El filtro pasa altas se implementa con un circuito **RC serie**, tomando la salida sobre la resistencia.

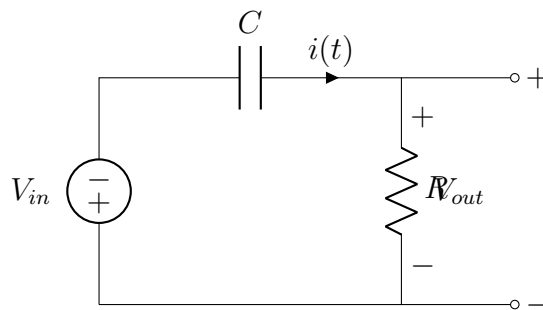


Figura 3: Circuito RC pasa altas (salida sobre R).

calculo de los componentes:

La frecuencia de corte del circuito RC está dada por:

$$f_c = \frac{1}{2\pi RC} \implies \omega_c = \frac{1}{RC} = 2\pi f_c \quad (24)$$

Seleccionando $C = 100$ nF (serie E24), se calcula la resistencia necesaria para obtener $f_c = 500$ Hz:

$$R = \frac{1}{2\pi f_c C} = \frac{1}{2\pi(500)(100 \times 10^{-9})} = \frac{1}{3.14159 \times 10^{-4}} = 3183.1 \, \Omega \approx 3.18 \, \text{k}\Omega \quad (25)$$



Se utiliza una resistencia de $R = 3.18 \text{ k}\Omega$ (valor teórico), lo cual resulta en una frecuencia de corte real de:

$$f_{c,\text{real}} = \frac{1}{2\pi(3180)(100 \times 10^{-9})} \approx 500 \text{ Hz} \quad (26)$$

2.2.1. Obtención de la función de transferencia $H(s)$

Aplicando la LVK al circuito RC en el dominio de Laplace:

$$V_{in}(s) = \frac{1}{sC} I(s) + R I(s) = I(s) \left(\frac{1}{sC} + R \right) \quad (27)$$

La salida se toma sobre la resistencia:

$$V_{out}(s) = R \cdot I(s) \quad (28)$$

La función de transferencia es:

$$H(s) = \frac{V_{out}(s)}{V_{in}(s)} = \frac{R}{\frac{1}{sC} + R} \quad (29)$$

Multiplicando numerador y denominador por sC :

$$H(s) = \frac{sRC}{sRC + 1} \quad (30)$$

Dividiendo entre RC :

$$\boxed{H(s) = \frac{s}{s + \omega_c}, \quad \omega_c = 2\pi(500) = 3141.59 \text{ rad/s}} \quad (31)$$

La ganancia es cero en DC ($H(0) = 0$, bloquea la componente continua) y unitaria en alta frecuencia ($\lim_{s \rightarrow \infty} H(s) = 1$), confirmando el comportamiento pasa altas.

2.2.2. Discretización mediante la transformada bilineal (Tustin)

Paso 1: Pre-distorsión de frecuencia.

$$\Omega_a = \tan \left(\frac{\pi f_c}{f_s} \right) = \tan \left(\frac{\pi \cdot 500}{8000} \right) = \tan \left(\frac{\pi}{16} \right) = 0.19891 \quad (32)$$

$$H(s) = \frac{s}{s + \Omega_a} \quad (33)$$



Paso 2: Sustitución bilineal $s = \frac{z-1}{z+1}$.

$$H(z) = \frac{\left(\frac{z-1}{z+1}\right)}{\left(\frac{z-1}{z+1}\right) + \Omega_a} \quad (34)$$

Simplificacion:

Multiplicando numerador y denominador por $(z+1)$:

$$H(z) = \frac{(z-1)}{(z-1) + \Omega_a(z+1)} \quad (35)$$

Expandiendo el denominador:

$$\begin{aligned} (z-1) + \Omega_a(z+1) &= z-1 + \Omega_a z + \Omega_a \\ &= z(1 + \Omega_a) + (\Omega_a - 1) \end{aligned} \quad (36)$$

Dividiendo numerador y denominador entre $(1 + \Omega_a)$:

$$H(z) = \frac{\frac{1}{1 + \Omega_a}(z-1)}{z + \frac{\Omega_a - 1}{1 + \Omega_a}} = \frac{\frac{1}{1 + \Omega_a}(z-1)}{z - \frac{1 - \Omega_a}{1 + \Omega_a}} \quad (37)$$

Paso 3: Identificación con la forma estándar. Comparando con la forma canónica $H(z) = \frac{A_0 z + A_1}{z - B_1}$:

$$A_0 = \frac{1}{1 + \Omega_a} = \frac{1}{1 + 0.19891} = \frac{1}{1.19891} = 0.8341 \quad (38)$$

$$A_1 = -\frac{1}{1 + \Omega_a} = -0.8341 \quad (39)$$

$$B_1 = \frac{1 - \Omega_a}{1 + \Omega_a} = \frac{1 - 0.19891}{1 + 0.19891} = \frac{0.80109}{1.19891} = 0.6682 \quad (40)$$

La función de transferencia discreta resulta:

$$\boxed{H(z) = \frac{0.8341 z - 0.8341}{z - 0.6682}} \quad (41)$$



2.2.3. Ecuación en diferencias

A partir de $H(z) = Y(z)/U(z)$:

$$Y(z) (z - B_1) = U(z) (A_0 z + A_1) \quad (42)$$

Dividiendo ambos lados entre z y aplicando la transformada z inversa:

$$y(k) - B_1 y(k-1) = A_0 u(k) + A_1 u(k-1) \quad (43)$$

Reorganizando:

$$\boxed{y(k) = A_0 u(k) + A_1 u(k-1) + B_1 y(k-1)} \quad (44)$$

Sustituyendo los valores numéricos:

$$y(k) = 0.8341 u(k) - 0.8341 u(k-1) + 0.6682 y(k-1) \quad (45)$$

coeficientes:

Tabla 2: Coeficientes del filtro pasa altas ($f_c = 500$ Hz, $f_s = 8000$ Hz).

Coeficiente	Expresión	Valor numérico
A_0	$\frac{1}{1 + \Omega_a}$	0.8341
A_1	$-\frac{1}{1 + \Omega_a}$	-0.8341
A_2	—	0.0
B_1	$\frac{1 - \Omega_a}{1 + \Omega_a}$	0.6682
B_2	—	0.0



1. **En DC** ($z = 1$):

$$H(1) = \frac{A_0 + A_1}{1 - B_1} = \frac{0.8341 - 0.8341}{1 - 0.6682} = \frac{0}{0.3318} = 0 \quad (46)$$

Esto confirma que el filtro pasa altas bloquea completamente la componente DC.

2. **En Nyquist** ($z = -1$):

$$H(-1) = \frac{A_0 - A_1}{1 + B_1} = \frac{0.8341 + 0.8341}{1 + 0.6682} = \frac{1.6682}{1.6682} = 1.0 \quad (47)$$

Esto confirma que la ganancia es unitaria en la frecuencia de Nyquist.

3. **Suma de coeficientes del numerador:**

$$A_0 + A_1 = 0.8341 - 0.8341 = 0 \quad (48)$$

garantiza $H(1) = 0$.

2.2.4. Diagrama de flujo de señal

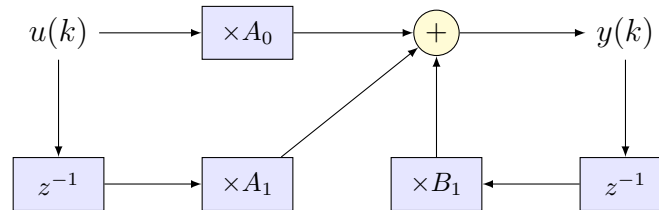


Figura 4: Diagrama de flujo de señal del filtro pasa altas (Forma Directa I).



2.3. Resumen de resultados de ambos filtros

En esta subsección se sintetizan los resultados finales obtenidos para cada filtro, de acuerdo con los cálculos y la implementación en el código.

Filtro pasa bajas

- Frecuencia de corte: $f_c \approx 79.6$ Hz, con $f_s = 8000$ Hz.
- Pre-distorsión: $\Omega_a = 0.03125$.
- Coeficientes: $A_0 = A_1 = 0.0303$, $B_1 = 0.9394$.
- Ecuación en diferencias: $y(k) = 0.0303 u(k) + 0.0303 u(k-1) + 0.9394 y(k-1)$.
- Verificación: $H(1) = 1$ (ganancia unitaria en DC).

Filtro pasa altas

- Frecuencia de corte: $f_c = 500$ Hz, con $f_s = 8000$ Hz.
- Pre-distorsión: $\Omega_a = 0.19891$.
- Coeficientes: $A_0 = 0.8341$, $A_1 = -0.8341$, $B_1 = 0.6682$.
- Ecuación en diferencias: $y(k) = 0.8341 u(k) - 0.8341 u(k-1) + 0.6682 y(k-1)$.
- Verificación: $H(1) = 0$ (bloqueo DC) y $H(-1) = 1$ (ganancia unitaria en Nyquist).



2.4. Implementación en MicroPython

2.4.1. Documentación del código

El programa comienza importando las bibliotecas necesarias para interactuar con el hardware de la Raspberry Pi Pico 2W. De la biblioteca `machine` se traen las clases `Pin`, `ADC` y `PWM`, que permiten configurar pines digitales, leer señales analógicas y generar ondas con modulación de ancho de pulso, respectivamente. También se importa el módulo `_thread`, que es la herramienta que ofrece MicroPython para trabajar con los dos núcleos del procesador RP2350 de forma simultánea, y el módulo `time`, que se utiliza para medir tiempos y hacer pausas con precisión de microsegundos. Además se importan `sys` y `select`, necesarios para implementar la lectura del puerto serial de forma no bloqueante.

A continuación se definen los parámetros fundamentales del sistema. La frecuencia de muestreo se fija en 8000 Hz, lo que implica que el filtro debe calcular una nueva muestra cada 125 microsegundos. Estos valores se guardan en las constantes `FS` y `PERIOD_US` para que sean fáciles de modificar si en algún momento se quisiera cambiar la tasa de muestreo. También se definen las constantes `PRINT_SKIP_LP` y `PRINT_SKIP_HP`, que indican cada cuántas muestras se envían datos al puerto serial para su visualización; esto es necesario porque imprimir cada una de las 8000 muestras por segundo desbordaría el buffer USB y provocaría que los comandos dejaran de responder.

Después viene la configuración del hardware. El pin GP26 se configura como entrada analógica a través de la clase `ADC`, ya que es uno de los pines que tiene convertidor analógico-digital integrado en la Pico. El pin GP3 se configura como salida PWM para generar la onda cuadrada que sirve como señal de prueba; se le asigna una frecuencia inicial de 50 Hz y un ciclo de trabajo del 50 %, lo que produce una onda simétrica.

Los coeficientes de los dos filtros se declaran como constantes globales. Para el filtro pasa bajas de primer orden, diseñado con $f_c \approx 79.6$ Hz, los coeficientes son $A_0 = A_1 = 0.0303$ y $B_1 = 0.9394$, obtenidos mediante la transformada bilineal como se mostró en la sección anterior. Para el filtro pasa altas con $f_c = 500$ Hz, los coeficientes son $A_0 = 0.8341$, $A_1 = -0.8341$ y $B_1 = 0.6682$, con $A_2 = 0$ y $B_2 = 0$. Estos valores se almacenan con los decimales necesarios para mantener una precisión adecuada en los cálculos de punto flotante.

El estado compartido entre los dos núcleos se almacena en una lista mutable de Python llamada `st`, que contiene once elementos: dos banderas de control (si el filtro está corriendo y



cuál filtro está activo), las variables de estado de ambos filtros (entradas y salidas anteriores), y tres valores para la comunicación con el Núcleo 0 (la última entrada leída, la última salida calculada, y una bandera que indica si hay datos nuevos listos para imprimir). Se eligió una lista mutable porque en el RP2350, el Núcleo 1 no puede acceder de manera confiable a las variables globales del módulo principal; al pasar la lista como argumento a la función del hilo, ambos núcleos pueden leer y escribir los mismos datos sin problema, ya que la lista se pasa por referencia. Para hacer el código más legible, se definen constantes con los nombres de cada índice de la lista.

La parte central del programa es la función `filter_core`, que se ejecuta en el Núcleo 1 del procesador. Esta función recibe como argumentos todo lo que necesita para operar: la lista de estado compartido, el objeto del ADC, el módulo de tiempo, todos los coeficientes de ambos filtros, el periodo de muestreo y el valor de salto de impresión. De esta forma, la función no hace ninguna referencia a nombres globales del módulo, lo que evita el error de visibilidad que tiene el RP2350 con los hilos. Dentro de un ciclo infinito, mientras la bandera de ejecución esté activa, la función registra el tiempo de inicio, lee el valor del ADC convirtiéndolo a voltaje entre 0 y 3.3 V, y evalúa la ecuación en diferencias del filtro seleccionado. Cada cierto número de muestras (determinado por `PRINT_SKIP_LP` o `PRINT_SKIP_HP`, según el filtro), guarda los valores de entrada y salida en la lista compartida y activa la bandera de datos nuevos para que el Núcleo 0 los imprima. Al final de cada iteración, calcula cuánto tiempo tardó el procesamiento y espera el tiempo restante para completar exactamente los 125 microsegundos del periodo de muestreo.

La función `filter_core` se lanza en el Núcleo 1 mediante la instrucción `_thread.start_new_thread`, pasando como argumentos la lista de estado, el objeto del ADC, el módulo de tiempo, los coeficientes de ambos filtros, el periodo y el salto de impresión. De este modo, ambos núcleos del RP2350 trabajan de manera simultánea sin interferirse: el Núcleo 1 se dedica exclusivamente al cálculo del filtro, y nunca accede al puerto serial.

El Núcleo 0 se encarga de toda la comunicación serial. Para poder recibir comandos y al mismo tiempo enviar los datos del filtro al Serial Plotter, se utiliza el mecanismo de `select.poll()`, que permite verificar si hay datos disponibles en la entrada estándar sin quedarse bloqueado esperando. El ciclo principal del Núcleo 0 hace tres cosas en cada iteración: primero revisa si llegó algún carácter por el puerto serial y, si es así, lo agrega a un buffer hasta recibir un salto de línea, momento en que procesa el comando completo; después revisa si la bandera de datos



nuevos está activa en la lista compartida y, si lo está, imprime los valores de entrada y salida en el formato **entrada,salida** que espera el Serial Plotter; y por último hace una pausa breve de 200 microsegundos antes de repetir. Los comandos disponibles son: **START** activa el filtrado; **STOP** lo detiene, reinicia las variables de estado del filtro activo y vuelve a mostrar el menú; **LP** selecciona el filtro pasa bajas y reinicia sus variables; **HP** hace lo mismo para el filtro pasa altas; **FREQ** seguido de un número cambia la frecuencia de la onda cuadrada de prueba; y **STATUS** muestra en pantalla qué filtro está seleccionado, si está activo o detenido, y las frecuencias del sistema y de la onda de prueba.

2.4.2. Código fuente

Listing 1: Código MicroPython del sistema de filtros digitales IIR para Raspberry Pi Pico 2W.

```
1 # =====
2 # filtros_iir.py
3 # Sistema de Filtros Digitales IIR - Raspberry Pi Pico 2W
4 #
5 # Filtro 1 (LP): Pasa Bajas de 1er Orden (Intacto)
6 # Filtro 2 (HP): Pasa Altas (Ajustado a los coeficientes de tu
   captura)
7 #
8 # fs = 8000 Hz | T = 125 us
9 # =====
10
11 from machine import Pin, ADC, PWM
12 import _thread
13 import time
14 import sys
15 import select
16
17 # =====
18 # PARAMETROS DEL SISTEMA
19 # =====
20 FS = 8000                # Frecuencia de muestreo (Hz)
21 PERIOD_US = 125          # Periodo de muestreo (us)
22
```



```
23 PRINT_SKIP_LP = 10
24 PRINT_SKIP_HP = 15
25
26 # =====
27 # CONFIGURACION DE HARDWARE
28 # =====
29 adc = ADC(Pin(26))          # GP26 = ADC0 (entrada analogica)
30 pwm_sq = PWM(Pin(3))        # GP3 = Onda cuadrada de prueba
31 pwm_sq.freq(50)             # Frecuencia inicial: 50 Hz
32 pwm_sq.duty_u16(32768)      # 50% duty cycle
33
34 # =====
35 # COEFICIENTES - FILTRO PASA BAJAS (1er Orden)
36 # =====
37 LP_A0 = 0.0303
38 LP_A1 = 0.0303
39 LP_B1 = 0.9394
40
41 # =====
42 # COEFICIENTES - FILTRO PASA ALTAS
43 # =====
44 HP_A0 = 0.8341
45 HP_A1 = -0.8341
46 HP_A2 = 0.0
47 HP_B1 = 0.6682
48 HP_B2 = 0.0
49
50 # =====
51 # ESTADO COMPARTIDO (Visible desde ambos cores)
52 # =====
53 st = [0, 0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0]
54
55 RUN = 0
56 FILT = 1
57 LU1 = 2
58 LY1 = 3
```




```
59 HU1  = 4
60 HU2  = 5
61 HY1  = 6
62 HY2  = 7
63 SU   = 8
64 SY   = 9
65 ND   = 10
66
67 # =====
68 # NUCLEO 1: ECUACION EN DIFERENCIAS
69 # =====
70 def filter_core(st, adc_obj, tm,
71                lp_a0, lp_a1, lp_b1,
72                hp_a0, hp_a1, hp_a2, hp_b1, hp_b2,
73                period, skip_lp, skip_hp):
74     counter = 0
75
76     while True:
77         # 1. Registro de tiempo exacto al inicio del ciclo
78         t0 = tm.ticks_us()
79
80         # 2. Ejecucion matematica solo si la bandera RUN esta activa
81         if st[0] == 1:
82             raw = adc_obj.read_u16()
83             u_k = (raw / 65535.0) * 3.3
84
85             if st[1] == 0:
86                 # ---- Filtro Pasa Bajas ----
87                 y_k = lp_a0 * u_k + lp_a1 * st[2] + lp_b1 * st[3]
88                 st[2] = u_k
89                 st[3] = y_k
90                 current_skip = skip_lp
91
92             else:
93                 # ---- Filtro Pasa Altas ----
```



```
94         y_k = hp_a0 * u_k + hp_a1 * st[4] + hp_a2 * st[5] +  
95             hp_b1 * st[6] + hp_b2 * st[7]  
96         st[5] = st[4]  
97         st[4] = u_k  
98         st[7] = st[6]  
99         st[6] = y_k  
100         current_skip = skip_hp  
101  
102         # 3. Peak Catcher dinamico  
103         counter += 1  
104         if counter >= current_skip or (st[1] == 1 and abs(y_k -  
105             st[9]) > 0.8):  
106             st[8] = u_k  
107             st[9] = y_k  
108             st[10] = 1  
109             counter = 0  
110  
111         # 4. Heartbeat de sincronizacion:  
112         # Garantiza el retraso exacto de 125us durante la ejecucion  
113         y al detenerse  
114         elapsed = tm.ticks_diff(tm.ticks_us(), t0)  
115         if elapsed < period:  
116             tm.sleep_us(period - elapsed)  
117  
118         # =====  
119         # INICIAR NUCLEO 1  
120         # =====  
121         _thread.start_new_thread(filter_core, (  
122             st, adc, time,  
123             LP_A0, LP_A1, LP_B1,  
124             HP_A0, HP_A1, HP_A2, HP_B1, HP_B2,  
125             PERIOD_US, PRINT_SKIP_LP, PRINT_SKIP_HP  
126         ))  
  
127         # =====  
128         # NUCLEO 0: COMUNICACION SERIAL NO BLOQUEANTE E IMPRESION
```



```
127 # =====
128 poll_obj = select.poll()
129 poll_obj.register(sys.stdin, select.POLLIN)
130
131 print("\n" + "=" * 50)
132 print("  Sistema de Filtros Digitales IIR")
133 print("  Raspberry Pi Pico 2W | fs = {} Hz".format(FS))
134 print("=" * 50)
135 print("Comandos disponibles:")
136 print("  START      - Iniciar filtrado")
137 print("  STOP       - Detener filtrado")
138 print("  LP        - Filtro pasa bajas")
139 print("  HP        - Filtro pasa altas (Ajustado a PDF)")
140 print("  FREQ <hz> - Frecuencia de onda cuadrada")
141 print("  STATUS     - Estado actual del sistema")
142 print("=" * 50)
143
144 cmd_buf = ""
145
146 while True:
147     if poll_obj.poll(0):
148         ch = sys.stdin.read(1)
149         if ch in ('\n', '\r'):
150             cmd = cmd_buf.strip().upper()
151             cmd_buf = ""
152
153             if not cmd:
154                 pass
155             elif cmd == "START":
156                 st[RUN] = 1
157                 print("OK: Filtrado iniciado")
158             elif cmd == "STOP":
159                 st[RUN] = 0
160                 if st[FILT] == 0:
161                     st[LU1] = 0.0
162                     st[LY1] = 0.0
```



```
163         else:
164             st[HU1] = 0.0
165             st[HU2] = 0.0
166             st[HY1] = 0.0
167             st[HY2] = 0.0
168             print("OK: Filtrado detenido")
169             print("=" * 50)
170             print("Comandos: START STOP LP HP FREQ <hz> STATUS")
171             print("=" * 50)
172         elif cmd == "LP":
173             st[RUN] = 0
174             st[FILT] = 0
175             st[LU1] = 0.0
176             st[LY1] = 0.0
177             print("OK: Pasa Bajas Seleccionado")
178         elif cmd == "HP":
179             st[RUN] = 0
180             st[FILT] = 1
181             st[HU1] = 0.0
182             st[HU2] = 0.0
183             st[HY1] = 0.0
184             st[HY2] = 0.0
185             print("OK: Pasa Altas Seleccionado")
186         elif cmd.startswith("FREQ"):
187             parts = cmd.split()
188             if len(parts) >= 2:
189                 try:
190                     freq = int(parts[1])
191                     pwm_sq.freq(freq)
192                     print("OK: Onda cuadrada = {} Hz".format(
193                         freq))
194                 except ValueError:
195                     print("ERR: Frecuencia no valida")
196             else:
197                 print("Uso: FREQ <frecuencia_hz>")
198         elif cmd == "STATUS":
```



```
198         nombre = "Pasa Bajas" if st[FILT] == 0 else "Pasa
199             Altas"
200         estado = "Activo" if st[RUN] == 1 else "Detenido"
201         print("Filtro: {} | Estado: {} | fs: {} Hz | Onda
202             Cuadrada: {} Hz".format(
203             nombre, estado, FS, pwm_sq.freq()))
204     else:
205         print("ERR: Comando desconocido: {}".format(cmd))
206     elif ch:
207         cmd_buf += ch
208
209     if st[ND] == 1:
210         st[ND] = 0
211         print("{:.4f},{:.4f}".format(st[SU], st[SY]))
212
213     time.sleep_us(200)
```

2.4.3. Diagrama de conexión física

Para probar el sistema, la señal de prueba generada por el pin GP3 (PWM) se conecta directamente al pin GP26 (ADC0) mediante un cable puente. De esta forma, la onda cuadrada producida por el propio microcontrolador entra al convertidor analógico-digital sin necesidad de componentes externos, lo que permite verificar el funcionamiento de los filtros digitales de manera sencilla.

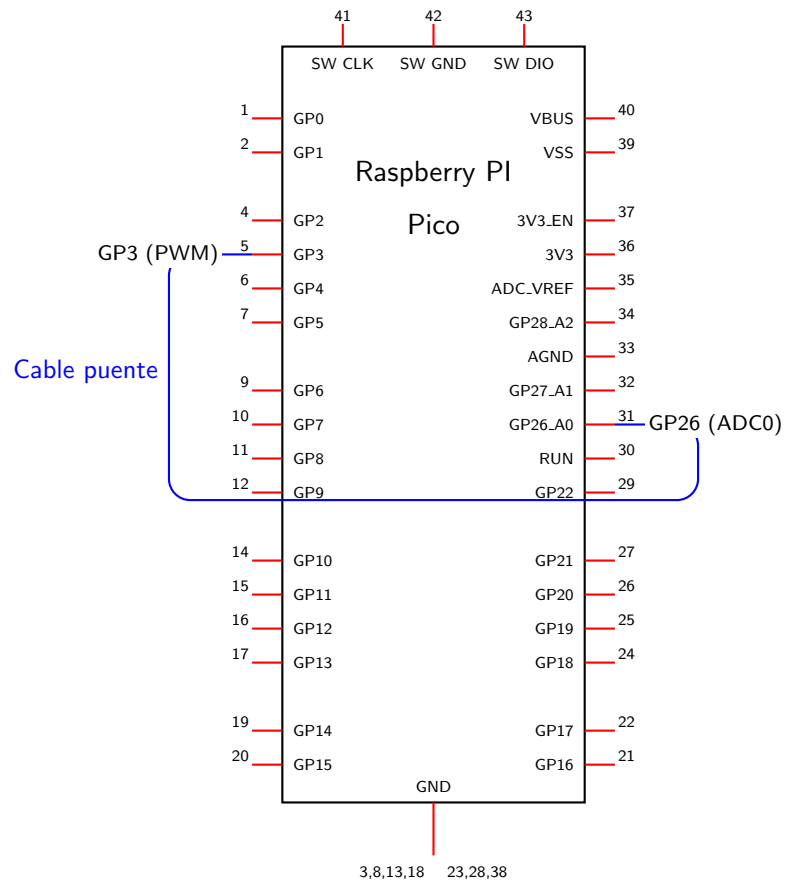


Figura 5: Conexión física: cable puente entre GP3 (salida PWM) y GP26 (entrada ADC) en la Raspberry Pi Pico 2W.



3. Resultados:

3.1. Coeficientes

Tabla 3: Comparación de coeficientes de los filtros diseñados.

Coeficiente	Pasa Bajas 1er Orden	Pasa Altas 2do Orden
A_0	0.16591	0.63897
A_1	0.16591	-1.27795
A_2	0.0	0.63897
B_1	0.66818	1.14305
B_2	0.0	-0.41286
f_c	500 Hz	800 Hz
Tipo	IIR Butterworth	RLC Butterworth
Orden	1	2



3.1.1. Valores esperados con señal de onda cuadrada

Al aplicar una onda cuadrada de frecuencia f_{sq} como señal de prueba, se esperan los siguientes comportamientos:

Tabla 4: Comportamiento esperado con onda cuadrada a distintas frecuencias.

f_{sq} (Hz)	Pasa Bajas ($f_c = 500$ Hz)	Pasa Altas ($f_c = 800$ Hz)
50 Hz	La salida sigue la onda cuadrada con esquinas redondeadas (atenuación mínima).	La salida muestra picos en cada transición, decayendo a cero entre flancos.
200 Hz	Onda cuadrada visible con suavizado moderado de las transiciones.	Picos pronunciados en los flancos; la componente fundamental se atenúa parcialmente.
500 Hz	Señal significativamente suavizada; la fundamental pasa con -3 dB.	Picos visibles; la fundamental pasa parcialmente (-8.9 dB de atenuación).

3.2. Filtro de primer orden

En esta sección se presentan los resultados obtenidos al aplicar el filtro pasa-bajas de primer orden con frecuencia de corte $f_c = 500$ Hz sobre una señal cuadrada evaluada a distintas frecuencias de entrada: 50, 200, 500 Hz.

En cada gráfica se muestra en azul la señal de entrada V_{in} (señal cuadrada) y en naranja la señal de salida V_{out} ya filtrada.

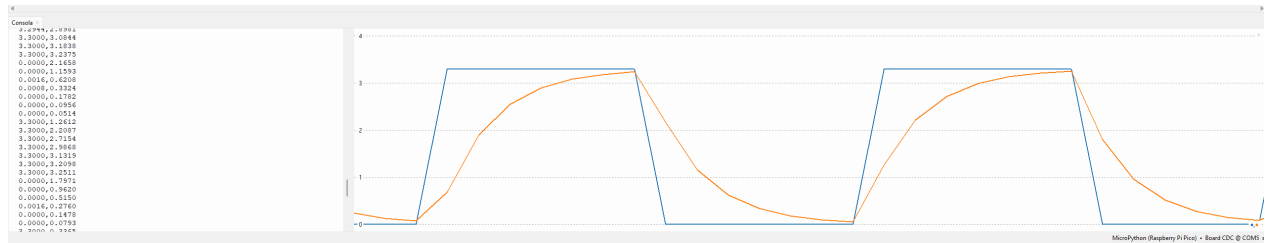


Figura 6: Filtro de primer orden a $f = 50$ Hz.

A 50 Hz, la frecuencia de entrada es muy inferior a la frecuencia de corte $f_c = 500$ Hz, por lo que la señal alterna lentamente y su forma cuadrada es perfectamente distinguible, con flancos abruptos y niveles estables en 3.3V y 0V. La señal de salida sigue de cerca a la entrada, presentando únicamente una ligera suavización en los flancos de subida y bajada. La amplitud se conserva prácticamente intacta, lo cual es consistente con que el filtro deja pasar sin atenuación significativa las componentes frecuenciales que se encuentran bien por debajo de f_c .

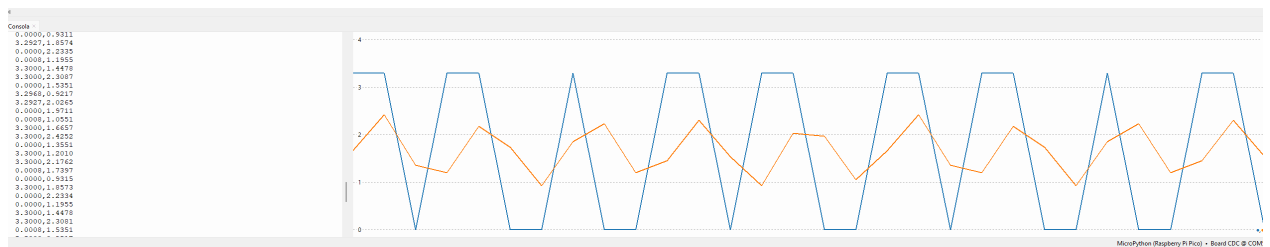


Figura 7: Filtro de primer orden a $f = 200$ Hz.

A 200 Hz la señal de entrada continúa siendo reconocible como cuadrada, aunque los ciclos son más rápidos. La salida presenta flancos más redondeados: el filtro comienza a atenuar las componentes armónicas de alta frecuencia que conforman los bordes abruptos de la señal

cuadrada, resultando en transiciones más graduales. La amplitud de la salida se mantiene cercana a la de la entrada, dado que 200 Hz aún se encuentra en la banda de paso del filtro.

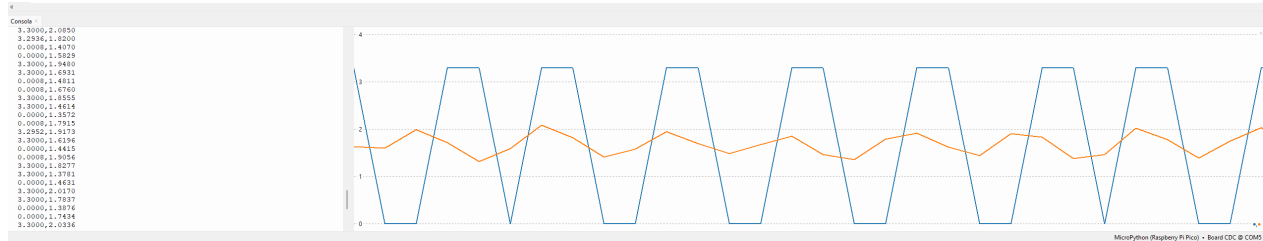


Figura 8: Filtro de primer orden a $f = 500$ Hz.

A 500 Hz, que corresponde exactamente a la frecuencia de corte f_c , la ganancia del filtro es $1/\sqrt{2} \approx 0.707$, equivalente a una atenuación de -3 dB. La señal de salida ha perdido amplitud de forma notable y sus flancos son considerablemente más suaves. La forma cuadrada ya no es distinguible con claridad; en cambio, la señal de salida adopta una apariencia triangular, producto de la atenuación de los armónicos superiores que componen la onda cuadrada.

3.3. Filtro pasa altas de segundo orden

Ahora se presentan los resultados obtenidos al aplicar el filtro pasa altas de segundo orden (Circuito RLC), con frecuencia de corte $f_c = 800$ Hz, sobre una señal cuadrada evaluada a distintas frecuencias de entrada como lo hicimos para el filtro anterior, 50, 200, 500 Hz.

Para comprender el comportamiento observado, es necesario recordar un principio fundamental, y es que la señal de entrada, al ser una onda cuadrada, está compuesta matemáticamente por la suma de múltiples señales senoidales con diferentes frecuencias, es decir, su frecuencia fundamental y una infinidad de armónicos. Es por esto que el filtro reacciona de manera distinta a ciertas partes de la señal, dejando pasar unas componentes frecuenciales y bloqueando otras. Asimismo, se notará en las gráficas que la señal de entrada, a pesar de ser cuadrada, llega a parecerse visualmente a una señal triangular en las frecuencias más altas debido a la enorme rapidez con la que alterna.

Igualmente en cada gráfica se muestra en azul la señal de entrada V_{in} (señal cuadrada) y en naranja la señal de salida V_{out} ya filtrada.

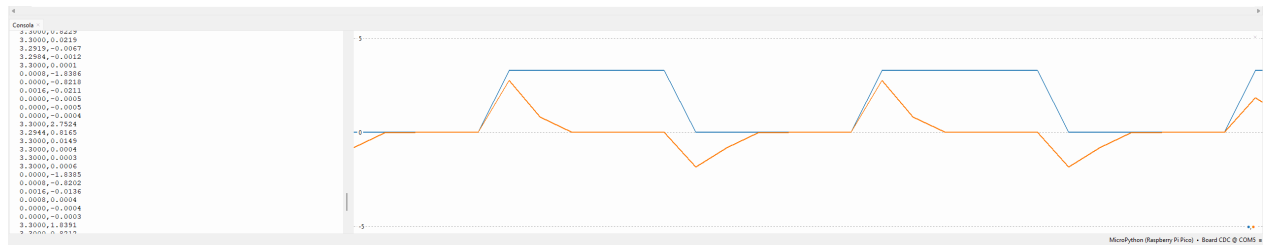


Figura 9: Filtro de segundo orden a $f = 50$ Hz.

A 50 Hz, la frecuencia fundamental de la señal se encuentra muy por debajo de la frecuencia de corte de 800 Hz. En la gráfica se observa que las partes planas de la onda cuadrada, que representan un comportamiento de voltaje constante (frecuencias cercanas a 0 Hz), son bloqueadas casi por completo. Sin embargo, en cada transición abrupta de voltaje (flancos de subida y bajada), la señal de salida presenta picos pronunciados. Esto ocurre porque dichos flancos están compuestos matemáticamente por una suma de armónicos de alta frecuencia, los cuales sí logran atravesar el filtro pasa altas. El resultado es un comportamiento puramente derivador, donde la señal reacciona solo a los cambios y decae a cero en los tramos estables.

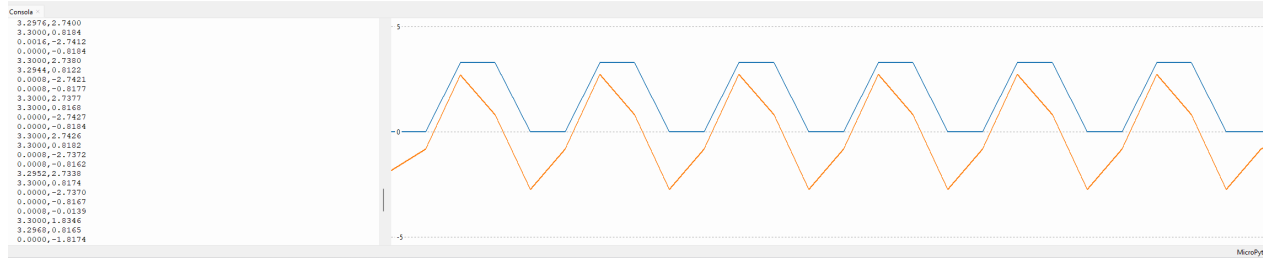


Figura 10: Filtro de segundo orden a $f = 200$ Hz.

A 200 Hz, la señal continúa en la banda de rechazo del filtro. Los picos generados por la alta frecuencia de los flancos siguen siendo la característica dominante de la señal de salida. No obstante, debido a que el periodo de la onda cuadrada es más corto, la respuesta transitoria de la señal filtrada no tiene el tiempo suficiente para decaer completamente a cero antes de que ocurra el siguiente flanco. A pesar de esto, la componente fundamental de 200 Hz sigue estando severamente atenuada.

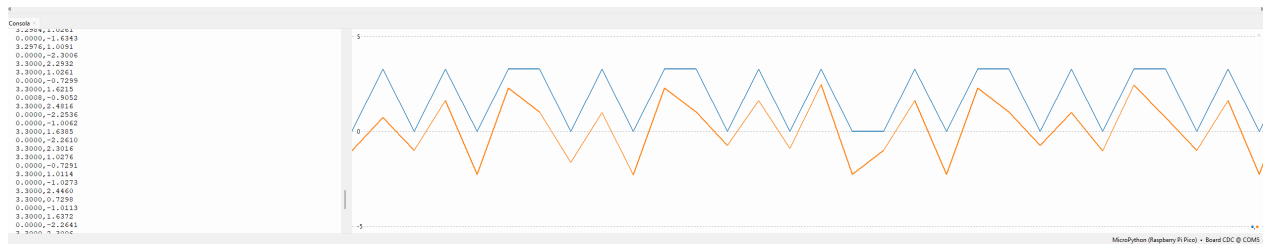


Figura 11: Filtro de segundo orden a $f = 500$ Hz.

A 500 Hz, nos aproximamos a la frecuencia de corte del filtro. Los picos correspondientes a los flancos se ensanchan visiblemente en la base, lo que indica que la componente fundamental comienza a pasar de manera parcial. De acuerdo con el análisis teórico, a esta frecuencia se presenta una atenuación de -8.9 dB. La señal de salida empieza a sostener la amplitud de forma más estable entre las transiciones, intentando replicar la forma de la onda cuadrada original, aunque con evidente distorsión.

3.4. Diferencias entre el Filtro de Primer Orden y Segundo Orden

Entre mayor es el orden de un filtro, mayor es la atenuación que ejerce sobre la señal; por ejemplo, en los filtros de Primer Orden la transición es más suave y lenta, mientras que en un filtro de Segundo Orden la transición es más breve y abrupta.



Aunque en esta práctica el filtro de Primer Orden es un pasa bajas y el de Segundo Orden es un pasa altas, podemos apreciar con claridad este comportamiento al observar la respuesta del sistema, donde el de segundo orden demuestra una mayor selectividad para rechazar las frecuencias no deseadas. Este comportamiento también se ve reflejado en el dominio digital al obtener la ecuación en diferencias. En un filtro de primer orden, la salida depende únicamente de una muestra anterior de la entrada ($u(k-1)$) y una de la salida ($y(k-1)$), de forma que para realizar los cálculos el sistema solo toma en cuenta un valor atrás ya procesado. En cambio, el filtro de segundo orden requiere el doble de información histórica ($k-1$ y $k-2$), lo que lo vuelve más complejo.



4. Conclusiones:

■ Espinoza Matamoros Percival Ulises:

A partir de la realización de esta práctica pude retomar conceptos vistos en asignaturas anteriores, de forma que a partir de diseñar un filtro y a partir de esta obtener sus funciones de transferencia, al tener que implementar el filtro en la Raspberry Pi Pico, pude aprender y comprender la importancia así como la necesidad de discretizar la función de transferencia, para así poder implementarla en un microcontrolador, donde por medio de una señal cuadrada de entrada se pudo observar el comportamiento del filtro, comprobando así que el filtro se comporta de la forma esperada, es decir, que se atenúan las frecuencias bajas para el caso del filtro pasa altas, y se atenúan las frecuencias altas para el caso del filtro pasa bajas. Por otro lado al implementar estos dos filtros se pudo observar que el filtro de primer orden tiene una respuesta más lenta, es decir, que tarda más en estabilizarse, mientras que el filtro de segundo orden tiene una respuesta más rápida, es decir, que tarda menos en estabilizarse. De forma que en la raspberry pi pico se pueden implementar filtros digitales de forma sencilla, y con un buen desempeño, siempre y cuando se tenga una buena discretización de la función de transferencia y se realicen los cálculos correspondientes de forma correcta.

- **Flores Colin Victor Jaziel:** Gracias a esta práctica, pude comprobar cómo el orden de un filtro puede definir de manera crítica el comportamiento de la señal de salida y es que quedó evidenciado que, a mayor orden, la atenuación ejercida es superior: mientras el filtro de primer orden presenta una transición más suave y lenta, el de segundo orden ofrece una respuesta mucho más breve y abrupta y este fenómeno se manifestó claramente al observar la respuesta del sistema, a pesar de que en nuestro montaje experimental el filtro de primer orden se configuró como pasa bajas y el de segundo orden como pasa altas.

La implementación en MicroPython fue de suma importancia para facilitar el análisis de cómo el orden impacta directamente en la complejidad del algoritmo y cuando obtuvimos las ecuaciones en diferencias, observé que un filtro de primer orden es computacionalmente ligero, pues solo requiere sólo un valor atrás tanto de la entrada como de la salida ($u(k-1)$ y $y(k-1)$). En contraste, el de segundo orden demanda el doble de información histórica y operaciones matemáticas adicionales. Esto me permitió concluir que el diseño de filtros digitales depende de las necesidades que hay que resolver y los



recursos de procesamiento disponibles en el microcontrolador.

■ **García Cortés Adolfo de Jesús:**

Esta práctica ayudó a repasar lo aprendido en clases pasadas sobre señales y a aplicarlo en un dispositivo real. Al diseñar los filtros RC, quedó claro que para pasarlos de la teoría analógica a la Raspberry Pi Pico, era obligatorio convertirlos a formato digital mediante matemáticas (usando la transformada bilineal). De esta forma se obtuvo la ecuación necesaria para programarlos.

Probar los filtros con una señal cuadrada fue muy útil porque, por su naturaleza, este tipo de señal está formada por la suma de muchas frecuencias distintas. Esto dejó ver claramente qué hace cada filtro: el pasa-bajas dejó pasar las señales lentas y frenó las rápidas, mientras que el pasa-altas hizo exactamente lo contrario.

Al comparar los filtros, se notó una diferencia importante: el filtro de primer orden bloquea las frecuencias no deseadas de una forma suave, pero el de segundo orden las corta de manera mucho más brusca. Esto también afecta la programación, ya que el de primer orden es más sencillo al solo necesitar el valor anterior de la entrada y la salida ($u(k-1)$ y $y(k-1)$). En cambio, el de segundo orden necesita los dos valores anteriores, lo que le exige más cálculos a la placa.

Por último, quedó demostrado que la Raspberry Pi Pico funciona muy bien para crear filtros digitales, siempre y cuando las conversiones matemáticas y la velocidad para tomar los datos sean las correctas. Los resultados físicos fueron iguales a lo que decía la teoría, lo que confirma que tanto el diseño como el código en MicroPython se hicieron bien.

- **Lara Hernandez Angel Husiel:** La realización de esta práctica me permitió integrar los conceptos teóricos de los filtros analógicos RC con las herramientas de procesamiento digital de señales. Al implementar los algoritmos en la Raspberry Pi Pico, comprendí que la transformación de una función de transferencia del dominio de Laplace al dominio discreto mediante la transformada bilineal es un paso fundamental para que el microcontrolador pueda interpretar el comportamiento físico deseado y esto lo puede apreciar de mejor manera durante las pruebas con la señal cuadrada donde note cómo cada filtro extrae componentes distintas de la señal: el suavizado de las transiciones en el pasa bajas y el aislamiento de los flancos en el pasa altas.



Después de obtener las gráficas y hacer la comparación entre el primer y segundo orden pude visualizar la atenuación que ofrece cada tipo de filtro. Pude constatar que, aunque el filtro de segundo orden es más eficiente para aislar bandas de frecuencia específicas gracias a su corte más drástico, requiere una programación más cuidadosa de las ecuaciones en diferencias para asegurar la estabilidad del sistema. En esta práctica aprendí que un buen diseño matemático con una buena correcta implementación en hardware permite emular componentes analógicos con gran precisión, aprovechando la flexibilidad que ofrecen herramientas como Thony de MicroPython.

■ **Lugo Manzano Rodrigo:**

A lo largo de esta práctica, pude recordar lo ya aprendido en algunas asignaturas pasadas y ver cómo la teoría de circuitos analógicos se traslada al mundo digital mediante herramientas matemáticas. Pude observar cómo la señal cuadrada de entrada responde a los filtros dependiendo de su frecuencia, notando claramente cómo el pasa bajas suaviza la onda al eliminar los armónicos rápidos, mientras que el pasa altas aísla los cambios bruscos de los flancos. También fue evidente que subir a un segundo orden resulta en un corte de frecuencias mucho más estricto frente a la transición suave del primer orden, sin embargo, un reto y aprendizaje importante también estuvo en la implementación física. Trasladar las ecuaciones en diferencias al microcontrolador requirió de una buena optimización para no perder el ritmo de la frecuencia de muestreo establecida. Para lograrlo, fue importante aprovechar la arquitectura de doble núcleo de la Raspberry Pi Pico 2W. Al aislar la ejecución matemática en el Núcleo 1, garantizamos que el filtro calculara cada muestra exactamente cada $125\mu s$, mientras que el Núcleo 0 se encargaba de la comunicación serial de forma no bloqueante. Esto me demostró que en el diseño no basta con tener un modelo matemático perfecto, sino que la eficiencia del código, la sincronización de los núcleos y el manejo adecuado de los recursos del hardware igual son partes importantes para lograr un procesamiento de señales digitales estable.



Referencias

- Butterworth, S. (1930). On the Theory of Filter Amplifiers. *Wireless Engineer*, 7(6), 536-541.
- MicroPython Contributors. (2024a). *_thread — Multithreading Support*. https://docs.micropython.org/en/latest/library/_thread.html
- MicroPython Contributors. (2024b). *class ADC — Analog to Digital Conversion*. <https://docs.micropython.org/en/latest/library/machine.ADC.html>
- MicroPython Contributors. (2024c). *class PWM — Pulse Width Modulation*. <https://docs.micropython.org/en/latest/library/machine.PWM.html>
- MicroPython Contributors. (2024d). *machine — Functions Related to the Hardware*. <https://docs.micropython.org/en/latest/library/machine.html>
- MicroPython Contributors. (2024e). *time — Time Related Functions*. <https://docs.micropython.org/en/latest/library/time.html>
- Ogata, K. (2010). *Modern Control Engineering* (5.^a ed.). Pearson.
- Python Software Foundation. (2024). *_thread — Low-Level Threading API*. https://docs.python.org/3/library/_thread.html
- Raspberry Pi Ltd. (2024a). *Raspberry Pi Pico 2 W Datasheet*. <https://datasheets.raspberrypi.com/pico/pico-2-w-datasheet.pdf>
- Raspberry Pi Ltd. (2024b). *RP2350 Datasheet: A Microcontroller by Raspberry Pi*. <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
- Sedra, A. S., Smith, K. C., Carusone, T. C., & Gaudet, V. (2020). *Microelectronic Circuits* (8.^a ed.). Oxford University Press.